



SewerStudio KI 4.1

*Ein Ein-Mann-Projekt: Was ein Nicht-Programmierer mit KI-
Unterstuetzung erschaffen kann*

Vollstaendiges System-Audit

Autor: Pascal Aschwanden

Datum: 12. April 2026

Version: 4.1

Plattform: WPF / .NET 8+ / Windows 11

VERTRAULICH — TECHNISCHES AUDIT-DOKUMENT

Inhaltsverzeichnis

1. Zusammenfassung (Executive Summary)

2. System-Architektur

3. Hardware-Uebersicht

4. Vollstaendige Feature-Liste

5. KI-Pipeline im Detail

6. Entwicklungsgeschichte (Timeline)

7. Architektur-Entscheidungen

8. Audit-Ergebnisse

9. Optimierungs-Roadmap

10. Projekt-Statistiken

KAPITEL 1

Zusammenfassung (Executive Summary)

Was SewerStudio ist und warum dieses Projekt bemerkenswert ist

Was ist SewerStudio?

SewerStudio KI 4.1 ist ein vollstaendiges Desktop-System fuer die automatisierte Kanalinspektion. Es verarbeitet Inspektionsvideos aus Kanaelen, erkennt Schaeden automatisch mittels Kuenstlicher Intelligenz und erstellt normkonforme Inspektionsprotokolle nach EN 13508-2 und dem Schweizer VSA-KEK-Standard (Ausgabe 2023).

Das Besondere an diesem Projekt: Es wurde von einem einzigen Entwickler erschaffen — einem gelernten Kanalinspekteur ohne formale Programmierausbildung. Pascal Aschwanden hat dieses System mit Hilfe von KI-Assistenten (Claude, ChatGPT, Gemini, DeepSeek) von Grund auf aufgebaut. Das Ergebnis ist ein professionelles, produktionsreifes System mit ueber 142.000 Zeilen C#-Code, ueber 790 Quelldateien und einer Multi-Modell KI-Pipeline, die vollstaendig lokal auf einer einzelnen Workstation laeuft.

Systemumfang auf einen Blick

32

Fenster & Dialoge

33+

ViewModels

140+

KI-Module & Services

13

CLI-Tools

5

Import-Formate

~3000

Inspektionsvideos

Architektur-Highlights

Clean Architecture

Vier klar getrennte Schichten: Domain, Application, Infrastructure und UI. Abhaengigkeiten fliessen nur nach innen. Geschäftslogik ist von Technologie entkoppelt.

Multi-Modell KI-Pipeline

Vier KI-Modelle arbeiten zusammen: YOLO fuer Objekterkennung, DINO fuer Grounding, SAM fuer Segmentierung und Qwen fuer Klassifikation. Alles orchestriert durch ein QualityGate-System.

Kern-Prinzip — Thin-AI: Die gesamte Geschäftslogik (Messungen, Protokollierung, Bewertung, Kostenberechnung, Hydraulik) laeuft in determinischem C#-Code. Das LLM (Qwen) wird ausschliesslich fuer die Textgenerierung der VSA-Code-Klassifikation verwendet. Keine kritischen Entscheidungen haengen von probabilistischem KI-Output ab.

KAPITEL 2

System-Architektur

Schichtenmodell, KI-Pipeline, VRAM-Management und Eskalationslogik

2.1 Schichtenarchitektur (Clean Architecture)

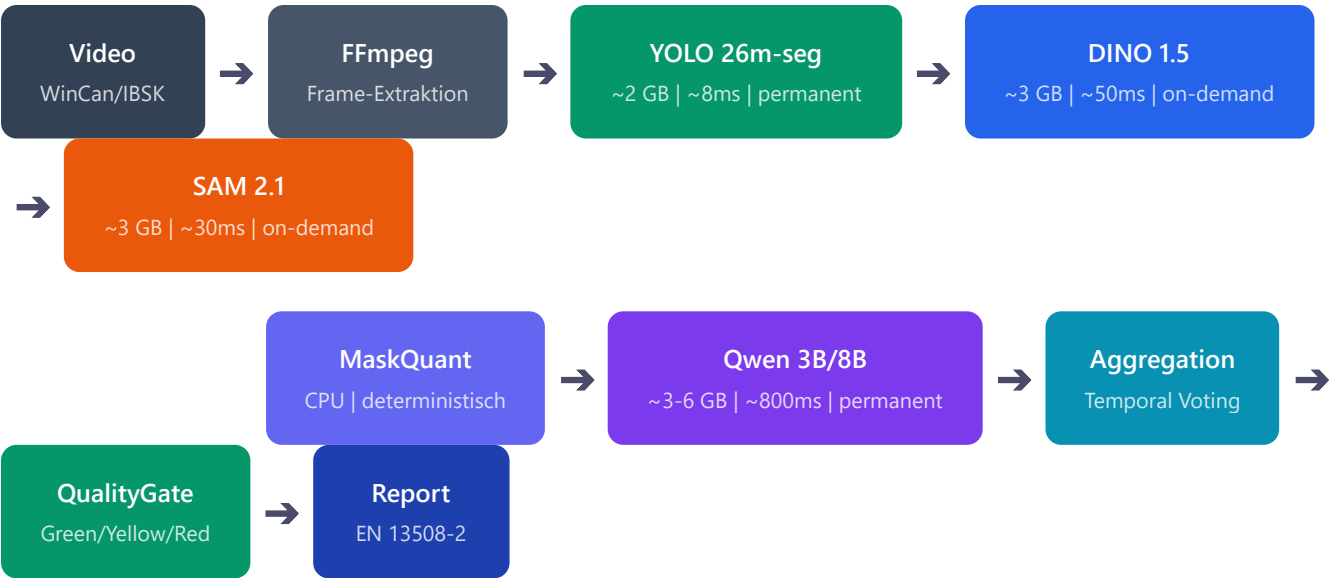
SewerStudio folgt dem Clean-Architecture-Muster mit vier klar getrennten Schichten. Abhaengigkeiten zeigen ausschliesslich nach innen — die aeusseren Schichten kennen die inneren, aber nicht umgekehrt.



SCHICHT	PROJEKT	VERANTWORTUNG	BEISPIELE
Domain	AuswertungPro.Next.Domain	Geschaeftsregeln, Entitaeten	HaltungRecord, ProtocolEntry, VsaCode
Application	AuswertungPro.Next.Application	Use Cases, Interfaces	IProtocolService, IExportServices, IImportServices
Infrastructure	AuswertungPro.Next.Infrastructure	Technische Implementierung	PdfImportService, WinCanDbImportService, CsvExcelExportService
UI	AuswertungPro.Next.UI	Darstellung, Benutzerinteraktion	PlayerWindow, DataPage, CodingModeWindow

2.2 KI-Pipeline — Vollstaendiger Datenfluss

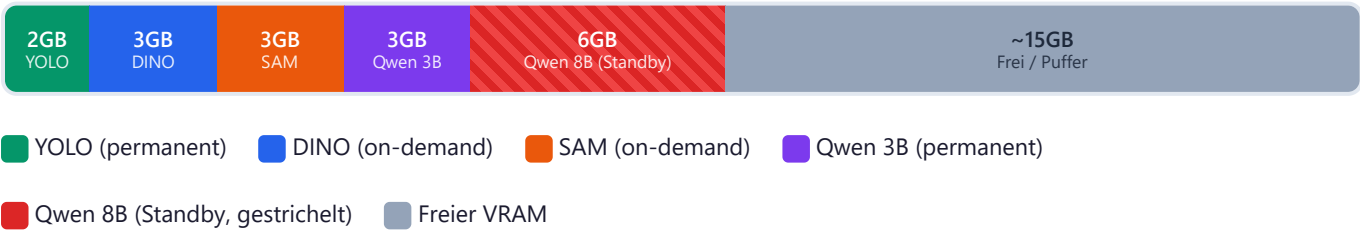
Die KI-Pipeline verarbeitet jedes Inspektionsvideo in einer definierten Kette von Modellen. Jedes Modell hat eine spezifische Aufgabe, und der QualityGate-Mechanismus entscheidet, ob zusaetzliche Modelle hinzugezogen werden muessen.



MODELL	AUFGABE	VRAM	LATENZ	RESIDENCY	RUNTIME
YOLO 26m-seg	Objekt-Detection + Instanz-Segmentierung	~2 GB	~8 ms	Permanent	TensorRT FP16
Grounding DINO 1.5 Edge	Text-Grounding, Uhrlage-Lokalisierung	~3 GB	~50 ms	On-Demand	Python Sidecar
SAM 2.1 Hiera-L	Praezise Maskensegmentierung	~3 GB	~30 ms	On-Demand	Python Sidecar
Qwen 2.5-VL 3B	VSA-Code-Klassifikation (primaer)	~3 GB	~800 ms	Permanent	Ollama
Qwen 2.5-VL 8B	VSA-Code-Klassifikation (Eskalation)	~6 GB	~1.5 s	Standby	Ollama
nomic-embed-text	Text-Embeddings fuer Knowledge Base	~0.3 GB	~5 ms	Permanent	Ollama

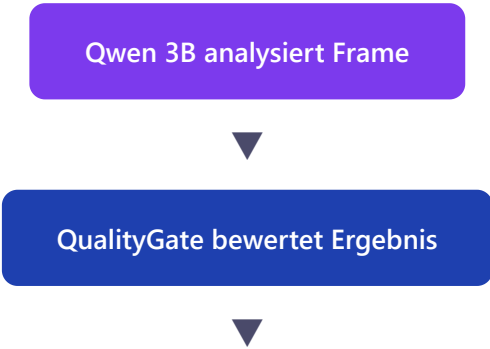
2.3 VRAM-Belegungsdiagramm (RTX 5090 — 32 GB)

Die folgende Darstellung zeigt die typische VRAM-Belegung waehrend der vollstaendigen Analyse. Das System ist so konzipiert, dass nie mehr als 29 GB gleichzeitig belegt werden.



2.4 Eskalations-Flussdiagramm

Das QualityGate bewertet jeden Fund anhand gewichteter Signale (Confidence, Konsistenz, KB-Match, Messgenauigkeit). Je nach Ergebnis werden verschiedene Pfade eingeschlagen.





GATE	BEDINGUNG	AKTION	VRAM-AUSWIRKUNG
GREEN	Score ≥ 0.80 , KB-Match vorhanden, min. 2 Signale	Auto-Accept, in Knowledge Base speichern	Kein zusaetzlicher VRAM
YELLOW	Score 0.50–0.79 oder fehlende Signale	Eskalation zu Qwen 8B fuer Zweitanalyse	+6 GB (Qwen 8B laden)
RED	Score < 0.50 oder widerspruechliche Signale	DINO + SAM laden, detaillierte Segmentierung, Review Queue	+6 GB (DINO + SAM laden)

2.5 Inference-Orchestrator — GPU-Zustaende

Der `InferenceOrchestratorService` verwaltet drei explizite Zustaende, die bestimmen, welche Modelle zu einem Zeitpunkt im VRAM geladen sind.

ZUSTAND	GPU-MODELLE	CPU-TASKS	UEBERGANG
DETECT	YOLO	Tracker + Aggregator	QualityGate Yellow/Red → SEGMENT
SEGMENT	YOLO + SAM	MaskQuantification	Segmentierung fertig → CLASSIFY
CLASSIFY	YOLO + Qwen	Report-Generierung	Klassifikation fertig → DETECT

KAPITEL 3

Hardware-Uebersicht

Workstation-Spezifikationen und Nutzungsanalyse

3.1 GPU — NVIDIA GeForce RTX 5090

PARAMETER	WERT	GENUTZT	BEMERKUNG
VRAM	32 GB GDDR7	✓ ~17-27 GB	Budget: max. 29 GB stabil
CUDA Cores	21.760	✓	YOLO TensorRT FP16
Tensor Cores	680 (5. Gen)	✓	Qwen Inferenz via Ollama
RT Cores	170 (4. Gen)	X	Nicht relevant fuer KI
Speicherbandbreite	1792 GB/s	✓	Kritisch fuer LLM-Inferenz
TDP	575W	✓	Volle Last bei Parallel-Inferenz
NVLink	Nein (Single-GPU)	—	Multi-GPU nicht geplant

3.2 CPU — Intel Core Ultra 9 285K

PARAMETER	WERT	GENUTZT	BEMERKUNG
P-Cores	8x Lion Cove (bis 5.7 GHz)	✓	WPF UI, Orchestrierung
E-Cores	16x Skymont (bis 4.6 GHz)	✓	Background-Tasks, FFmpeg
Threads	24 (P-Cores: kein HT)	✓	Temporal Aggregation, Tracker
NPU	NPU 4 (13 TOPS)	X	Pre-Screening auf CPU schneller
Cache	36 MB L3	✓	Datenstrukturen, Embeddings
iGPU	Intel Arc (4 Xe2)	X	Nicht genutzt

3.3 Arbeitsspeicher & Storage

KOMPONENTE	SPEZIFIKATION	NUTZUNG
RAM	64 GB DDR5-6000 (2x32 GB)	WPF-App, Video-Frames, Embedding-Cache
Primaer-SSD	NVMe Gen5 (System + App)	Model-Loading, KB-Zugriff
Daten-SSD	NVMe Gen4 (Videodaten)	~3000 Inspektionsvideos

Hardware-Fazit: Die RTX 5090 mit 32 GB VRAM und 1792 GB/s Bandbreite ist der entscheidende Faktor fuer die lokale Multi-Modell-Inferenz. Der Core Ultra 9 285K liefert genuegend CPU-Kerne fuer die parallele Orchestrierung. NPU und iGPU bleiben ungenutzt, da die Workloads entweder zu schnell fuer NPU-Overhead sind oder direkt auf der dedizierten GPU laufen.

KAPITEL 4

Vollstaendige Feature-Liste

Alle Funktionsbereiche von SewerStudio im Ueberblick

4.1 Video & Codierung



Video-Player (LibVLC)

Vollstaendiger Video-Player mit Seek-Leiste, Frame-Navigation, Zeitstempel-Synchronisation und OSD-Meter-Erkennung. Unterstuetzt alle gaengigen Formate inkl. AVI, MP4, MPEG.



Live-Detection

Echtzeit-Schadenserkennung waehrend der Videowiedergabe. YOLO-Bounding-Boxes werden direkt auf dem Videobild ueberlagert. Confidence-Scores live angezeigt.



Coding Mode (8 Overlay-Tools)

Manueller Codierungsmodus mit 8 Werkzeugen: Distanz-Messung, Winkel, Uhrlage, Ausdehnung, Polygonflaeche, Rechteck, Punkt und Freihand-Linie. Alle Messungen werden in mm umgerechnet.



Quick Scan

Schnelle Voranalyse eines Videos mit reduzierter Frame-Rate. Identifiziert Stellen mit potentiellen Schaeden fuer die anschliessende detaillierte Analyse.



Full Analysis Pipeline

Vollstaendige automatische Analyse mit allen KI-Modellen: YOLO → DINO → SAM → Qwen. Inklusive QualityGate, Temporal Aggregation und automatischer Protokollerstellung.



Live-Frame-Fenster

Separates Fenster das den aktuellen Frame mit allen KI-Overlays (Bounding Boxes, Masken, Messwerte) in hoher Aufloesung anzeigt. Ideal fuer den zweiten Monitor.

4.2 Protokoll-Management



Beobachtungskatalog (EN 13508-2)

Vollstaendiger Katalog aller EN 13508-2 BeobachtungsCodes mit Beschreibungen, Quantifizierungen und VSA-Zuordnungen. Hierarchisch organisiert mit Such- und Filterfunktion.



VSA-Code-Explorer

Interaktiver Explorer fuer alle 27 VSA-Schadenskategorien. Zeigt fuer jeden Code: Beschreibung, Stufen 1-5, typische Bilder aus der Knowledge Base und Haeufigkeitsstatistiken.



Protokoll-Editor

Vollstaendiger Editor fuer Inspektionsprotokolle mit Inline-Bearbeitung. Unterstuetzt Hauptcode, Charakterisierungen (1. und 2.), Uhrlage, Quantifizierung und Freitext-Bemerkungen.



Protokoll-Versionshistorie

Vollstaendige Aenderungshistorie fuer jedes Protokoll. Jede Aenderung wird mit Zeitstempel, Benutzer und Quelle (manuell/KI) protokolliert. Rollback auf fruehere Versionen moeglich.

4.3 Import-Formate

FORMAT	SERVICE	BESCHREIBUNG
PDF	PdfImportService	Bestehende Inspektionsprotokolle im PDF-Format. Unterstuetzt Fretz AG, KIT Bauinspekt, Abwasser Uri und weitere Formate. Regex-basierte Felderkennung.
XTF	XtfImportService	Interlis-Transferformat der Schweiz (SIA 405). XML-basiert, enthaelt Geometrie und Attributdaten fuer Haltungen und Schaechte.
WINCAN	WinCanDbImportService	Access-Datenbanken (.mdb/.accdb) von WinCan VX. Haltungen, Inspektionen, Medien-Zuordnungen. Inkl. Video-Pfad-Mapping.
IBAK	IbakExportImportService	IBAK IKAS Exportformat. Strukturierte Inspektionsdaten mit Medienreferenzen und Codierungen.
KINS	KinsImportService	KINS-Austauschformat fuer Kanalinspektionsdaten. Standardisierter Datenaustausch zwischen verschiedenen Software-Systemen.

4.4 Export & Reporting



PDF-Export (QuestPDF)

Professionelle PDF-Berichte mit QuestPDF. Haltungsdossier mit Protokolltabelle, Schadenbilder, VSA-Bewertung und Zustandsklasse. Hydraulik-Bericht separat.



Excel-Export

Tabellarischer Export aller Inspektionsdaten im XLSX-Format. Filterbare Spalten, farbcodierte Zustandsklassen, Pivot-faehige Datenstruktur.



CSV-Export

Einfacher Textexport fuer Weiterverarbeitung in Drittsystemen. Konfigurierbare Trennzeichen und Spaltenauswahl.

4.5 Datenmanagement



Haltungen/Schacht DataGrid

Leistungsfaeihige Datentabellen fuer Haltungen und Schaechte. Sortierung, Filterung, Inline-Bearbeitung, Mehrfachauswahl und Kontextmenue. Virtualisiert fuer grosse Datenmengen.



Feld-Katalog-System

Konfigurierbarer Feldkatalog fuer alle Datenfelder. Benutzer koennen eigene Feldkataloge erstellen, importieren und zwischen Projekten teilen.



Auto-Save & Restore Points

Automatische Sicherungspunkte bei jeder wichtigen Aenderung. Wiederherstellung auf jeden gespeicherten Zustand. Schutz vor Datenverlust bei Absturz.



Medien-Konflikt-Center

Erkennung und Aufloesung von Konflikten bei der Video-/Foto-Zuordnung zu Haltungen. Batch-Suche fuer fehlende Medien, automatische Pfad-Korrektur.

4.6 Kosten & Sanierung



Kostenberechnung

Automatische Berechnung der Sanierungskosten basierend auf Schadensbild, Rohrgeometrie und Preiskatalog. Unterstuetzt CHF und EUR.



Preiskataloge

Verwaltung mehrerer Preiskataloge mit Positionen, Einheitspreisen und Mengengeruestberechnung. Import/Export als JSON.



Massnahmen-Templates

Vordefinierte Sanierungsmassnahmen (Inliner, Kurzliner, Roboter, Erneuerung) mit automatischer Zuordnung zu Schadenstypen und -stufen.



Angebots-Generator (Eigendevis)

Erstellung vollstaendiger Offerten mit Positionen, Mengen, Einheitspreisen und Gesamtkosten. Kombinierte Angebote ueber mehrere Haltungen. PDF-Druck.



KI-Sanierungs-Optimierung

KI-gestuetzte Optimierung der Sanierungsstrategie. Bewertet verschiedene Massnahmenkombinationen nach Kosten-Nutzen-Verhaeltnis und empfiehlt die optimale Variante.

4.7 Hydraulik



DWA-A 110 konforme Berechnung

Vollstaendige hydraulische Berechnung nach DWA-A 110 Standard. Beruecksichtigt Rohrgeometrie, Gefaelle, Rauheit und Wasserstand.



Teilfuellung / Vollfuellung

Berechnung der Leistungsfahigkeit bei Teil- und Vollfuellung. Grafische Darstellung der Fuellstands-Geschwindigkeits-Beziehung fuer verschiedene Profiltypen.



Ablagerungs-Berechnung

Berechnung der kritischen Fließgeschwindigkeit und Ablagerungsgefahr. Warnung bei Unterschreitung der Mindestgeschwindigkeit nach DWA-Richtlinie.

4.8 KI-Training & Selbstlernen



Training Center

Zentrale Verwaltung fuer alle Trainingsaktivitaeten. Video-Probe, Self-Training-Vergleiche, Meter-Timeline und GuidedVerification in einer Oberflaeche.



Self-Training Loop

Automatischer Kreislauf: KI analysiert → QualityGate bewertet → Green-Samples in KB → Few-Shots verbessern → KI wird besser. Kontinuierliche Verbesserung ohne manuelles Training.



Knowledge Base (SQLite + Embeddings)

Lokale Wissensdatenbank mit SQLite-Backend und nomic-embed-text Vektoren. Speichert verifizierte Samples fuer Few-Shot-Retrieval. Aktives Lernen und Qualitaetsueberwachung.



Benchmark Runner

Automatisierte Benchmark-Laeufe ueber definierte Haltungen. Misst Precision, Recall, F1-Score und VSA-Code-Treffer. Vergleich zwischen Modellversionen und Konfigurationen.



Active Learning

Priorisierung unsicherer Samples fuer manuelle Verifikation. Identifiziert unterrepraesentierete VSA-Codes und fokussiert das Training auf Schwachstellen.



Weight Re-Learning (Coordinate Descent)

Automatische Anpassung der QualityGate-Signalgewichte basierend auf Benchmark-Ergebnissen. Coordinate Descent Optimierung fuer optimale Green/Yellow/Red-Verteilung.

4.9 System & Werkzeuge



Dark/Light Theme

Umschaltbares Design zwischen dunklem und hellem Modus. Konsistente Farbpalette ueber alle 32 Fenster und Dialoge.



System Monitor

Echtzeit-Ueberwachung von CPU-Auslastung, GPU-VRAM, RAM-Verbrauch und Modell-Status. nvidia-smi Integration fuer GPU-Temperatur und Taktfrequenz.



13 CLI-Tools

Eigenstaendige Kommandozeilen-Werkzeuge:
PdfHeaderReader, DiagnosticPdfParser,
IbakPdfAnalyzer, QuickPdfAnalyzer, VsaPdfExtractor,
KnowledgeBaseInspector, MeasureCatalogCli,
PdfImageAnalyzer, AiDocPdf, MdbSchemaReader,
PdfProtocolParser und weitere.

KAPITEL 5

KI-Pipeline im Detail

Technische Spezifikation jedes Modells, Algorithmus und Service

5.1 YOLO 26m-seg — Objekterkennung & Instanzsegmentierung

YOLO ist das primaere Erkennungsmodell und laeuft permanent auf der GPU. Es erkennt Kanalschaeden in jedem Frame und liefert Bounding Boxes und Instanzmasken.

PARAMETER	WERT
Modell	YOLOv8/YOLO26 Medium-Segment (custom trained)
Runtime	TensorRT FP16 (NVIDIA)
Input-Groesse	640x640 px
VRAM	~1.5-2 GB
Latenz	~8 ms pro Frame
Confidence-Schwelle	0.30 (Y < 0.30 verwirft)
Klassen	Trainiert auf VSA-relevante Schadenskategorien
Residency	Permanent im VRAM
Tracking	Temporal Voting (statt ByteTrack/OC-SORT)
Output	BBox (x, y, w, h), Confidence, Klasse, Instanzmaske

5.2 Grounding DINO 1.5 Edge — Text-Grounding

DINO wird nur bei QualityGate Yellow/Red aktiviert. Es lokalisiert spezifische Objekte anhand von Textbeschreibungen (z.B. "crack at 3 o'clock position"). Laeuft im Python Sidecar.

PARAMETER	WERT
Modell	Grounding DINO 1.5 Edge
Runtime	Python FastAPI Sidecar (Port 8100)
VRAM	~3 GB
Latenz	~50 ms pro Frame
Residency	On-Demand (nur bei Yellow/Red)
Exklusiv mit	SAM 3 (werden zusammen geladen)

Typische Text-Prompts fuer DINO

- "crack" , "fracture" , "deformation"
- "root intrusion" , "deposit" , "obstruction"
- "joint displacement" , "connection" , "lateral"
- "corrosion" , "surface damage" , "infiltration"

Bekannter Bug N2: `VsaCodeResolver.NormalizeClock()` behandelt die englischen DINO-Ausgaben (top/bottom/left/right) nicht korrekt. Die Uhrfrage-Konvertierung erwartet deutsche Begriffe.

5.3 SAM 2.1 Hiera-L — Segmentierung

SAM erstellt praezise Pixel-genaue Masken fuer erkannte Schaeden. Die Masken werden fuer die MaskQuantification verwendet, die dann deterministische Messwerte (Flaeche in mm², Ausdehnung in mm) berechnet.

PARAMETER	WERT
Modell	SAM 2.1 Hiera-Large
Backbone	Hierarchical Vision Transformer
VRAM	~3 GB
Latenz	~30 ms pro Maske
Input	BBox-Prompts von YOLO oder DINO

PARAMETER	WERT
Output	Binaere Maske (H x W), IoU-Score
Residency	On-Demand, exklusiv mit DINO

Ring-Scan Verfahren

Fuer die Uhlage-Bestimmung wird ein Ring-Scan-Algorithmus verwendet: Die Maske wird in 12 Sektoren (entsprechend den Uhrzeiten 1-12) unterteilt. Fuer jeden Sektor wird der Pixel-Anteil berechnet. Der Sektor mit dem hoechsten Anteil bestimmt die primaere Uhlage, die Ausdehnung wird ueber die Nachbarsektoren ermittelt.

5.4 MaskQuantification — Deterministische Messung

Dieser Service wandelt KI-generierte Masken in physikalische Masse um. Vollstaendig deterministisch, kein LLM involviert. Das ist das Kern-Prinzip von Thin-AI.

MESSWERT	FORMEL / METHODE	EINHEIT
Schadensflaeche	$\text{Pixel-Count} \times (\text{Rohrdurchmesser} / \text{Bildbreite})^2$	mm ²
Schadensbreite	Max. horizontale Ausdehnung der Maske × Skalierungsfaktor	mm
Schadenshoehe	Max. vertikale Ausdehnung der Maske × Skalierungsfaktor	mm
Uhlage	Ring-Scan mit 12 Sektoren, gewichteter Schwerpunkt	1-12 Uhr
Ausdehnung	Winkelbereich der Maske im Querschnitt	Grad / Prozent
Umfangsanteil	Belegte Sektoren / 12 × 100	%

Bekannter Bug N8: PipeImageWidthRatio ist auf 0.70 hardcodiert. Bei Kameras mit anderem Sichtfeld kann die mm-Umrechnung ungenau sein. Schwer zu fixen ohne Referenzmessung.

5.5 Qwen 2.5-VL (3B / 8B) — VSA-Code-Klassifikation

Qwen ist das Vision-Language-Modell, das aus dem Kontext (Frame, Masken, Messwerte) den korrekten VSA-Code bestimmt. Es arbeitet in einer Dual-Model-Architektur: 3B als Primaermodell, 8B als Eskalation bei Yellow.

PARAMETER	QWEN 3B (PRIMAER)	QWEN 8B (ESKALATION)
Modell	qwen2.5-vl:3b	qwen2.5-vl:7b / 8b
Quantisierung	Q5_K_M	Q5_K_M
VRAM	~3 GB	~6 GB
Latenz	~800 ms	~1.5 s
KV-Cache	~2-3 GB/Slot	~3-4 GB/Slot
Residency	Permanent	Standby (lazy load)
Runtime	Ollama	Ollama
Output-Format	Strict JSON Schema	Strict JSON Schema

JSON-Schema fuer Qwen-Output

```
{
  "vsa_code": "BAB",           // VSA-Schadenskategorie
  "stufe": 3,                 // Schweregrad 1-5
  "uhrlage": "3",             // Uhrlage 1-12
  "ausdehnung": "3-5",        // Von-Bis Uhrlage
  "charakterisierung_1": "A",  // Erste Charakterisierung
  "charakterisierung_2": "",   // Zweite Charakterisierung
  "bemerkung": "...",         // Freitext
  "confidence": 0.85          // Modell-Confidence
}
```

Few-Shot Retrieval

Fuer jeden Frame werden die 3-5 aehnlichsten Samples aus der Knowledge Base geholt (via nomic-embed-text Cosine Similarity). Diese werden als Few-Shot-Beispiele in den Qwen-Prompt eingefuegt. So lernt das Modell aus frueher verifizierten Klassifikationen.

5.6 Temporal Aggregation — ActiveFindingState

Da ein Schaden typischerweise ueber mehrere Frames sichtbar ist, muessen die Frame-weisen Detektionen zu einem einzelnen Befund aggregiert werden. Der DetectionAggregator verwaltet dafuer ActiveFindingStates.

KONZEPT	BESCHREIBUNG
ActiveFindingState	Repraesentiert einen laufenden Befund ueber mehrere Frames. Sammelt Detektionen, Confidence-Werte und Positionen.
Temporal Voting	Statt Tracker (ByteTrack) wird ueber die letzten N Frames abgestimmt. Ein Fund gilt als bestaetigt wenn er in $\geq 60\%$ der Frames erkannt wurde.
Meter-Korrelation	OSD-Meterzahl wird ausgelesen und mit Fund-Position korreliert. Jeder Befund erhaelt eine praezise Meter-Angabe.
FindingKey	Eindeutiger Schluessel pro Befund: <code>VSA-Code UhrLage Meter</code> . Verhindert Duplikate.

Bekannter Bug N4: FindingKey `Code|Clock` ohne Meter kann bei Mehrfach-Schaeden gleichen Typs an gleicher Uhrlage kollidieren. Selten, aber moeglich.

5.7 QualityGate — Automatische Qualitaetsbewertung

Das QualityGate ist das zentrale Entscheidungssystem. Es bewertet jeden Fund anhand gewichteter Signale und entscheidet ueber den weiteren Verarbeitungspfad.

SIGNAL	GEWICHT	BESCHREIBUNG
YOLO Confidence	0.25	Erkennungs-Sicherheit des Detektionsmodells
Qwen Confidence	0.25	Klassifikations-Sicherheit des LLM
KB-Match Score	0.20	Aehnlichkeit zu verifizierten Samples in der Knowledge Base
Temporal Consistency	0.15	Konsistenz der Erkennung ueber mehrere Frames
Measurement Plausibility	0.15	Physikalische Plausibilitaet der Messwerte

GATE	SCORE-BEREICH	AKTION
GREEN	≥ 0.80	Auto-Accept, in KB speichern, kein Review noetig
YELLOW	0.50 – 0.79	Eskalation zu Qwen 8B, ggf. DINO+SAM fuer bessere Masken
RED	< 0.50	In Review Queue, manueller Check erforderlich

Bekannter Bug N9: Bei fehlenden Signalen wird renormalisiert, d.h. ein einzelnes Signal kann theoretisch Green erzeugen. Fix empfohlen: Minimum 2 Signale fuer Green.

5.8 Knowledge Base — SQLite + Embeddings

Die Knowledge Base ist das Langzeitgedaechtnis des Systems. Sie speichert verifizierte Klassifikationen als Referenz fuer zukuenftige Analysen.

KOMPONENTE	TECHNOLOGIE	FUNKTION
Datenspeicher	SQLite	Samples mit VSA-Code, Metadaten, Confidence, Verifikations-Status
Embedding-Modell	nomic-embed-text (Ollama)	768-dimensionale Textvektoren fuer Similarity Search
Retrieval	Cosine Similarity	Top-K aehnliche Samples fuer Few-Shot-Prompting
Qualitaetssicherung	KbQualityService	Ueberwachung der Verteilung, Erkennung von Bias
Auto-Approval	AutoApprovalService	Green-Samples mit Score ≥ 0.60 werden automatisch aufgenommen
Feedback Loop	FeedbackIngestionService	Manuelle Korrekturen fließen als neue Samples zurueck

KAPITEL 6

Entwicklungsgeschichte

Von der Idee zum vollstaendigen System in vier Monaten

Januar 2026

Phase 1 — Manuelle Codierung (Baseline)

Aufbau der Grundarchitektur: WPF-Anwendung mit MVVM-Pattern, Clean Architecture mit vier Schichten. Video-Player mit LibVLC, manuelle Codierung von Kanalschaeden nach EN 13508-2. Import von PDF-Protokollen und WinCan-Datenbanken. Erste Version des Haltungs-DataGrids. Alles rein manuell — keine KI-Unterstuetzung bei der Codierung, nur bei der Programmierung.

Ueberlegung: Zuerst ein stabiles Fundament schaffen, bevor KI hinzugefuegt wird. Die manuelle Codierung dient als Referenz (Ground Truth) fuer spaetere KI-Bewertung.

Februar 2026

Phase 2 — KI-Pipeline v1 (YOLO + DINO + SAM + Qwen 8B)

Integration der ersten KI-Pipeline: YOLO fuer Schadenserkennung, Grounding DINO fuer Lokalisierung, SAM fuer Segmentierung, Qwen 8B fuer VSA-Code-Klassifikation. Python Sidecar fuer DINO/SAM, Ollama fuer Qwen. Erste automatische Protokollerstellung. Erster Benchmark gegen manuelle Codierung.

Ueberlegung: Multi-Modell-Ansatz statt eines einzelnen grossen Modells. Jedes Modell hat eine klar definierte Aufgabe. Thin-AI-Prinzip: LLM nur fuer Textgenerierung, nicht fuer Messungen.

Maerz 2026

Phase 3 — Self-Learning + Dual-Model + QualityGate

Einfuehrung des QualityGate-Systems (Green/Yellow/Red), Knowledge Base mit Embeddings, Self-Training Loop, Dual-Model-Architektur (8B primaer + 32B Eskalation, spaeter auf 3B+8B umgestellt). Weight Re-Learning fuer QualityGate-Gewichte. Active Learning und Review Queue. Benchmark-Runner fuer systematische Qualitaetsmessung.

Ueberlegung: Das System muss sich selbst verbessern koennen. Green-Samples in die KB, Few-Shots werden besser, Precision steigt. Eskalation spart VRAM fuer 90% der Faelle.



April 2026

Phase 4 — V4.1 (Qwen 3B Primary, Optimierung, Audit)

Umstellung auf Qwen 3B als Primaermodell (statt 8B) zur VRAM-Einsparung und Latenz-Reduktion. Planung fuer parallele Qwen-Slots (OLLAMA_NUM_PARALLEL=2). Umfassendes System-Audit durch mehrere KI-Systeme (ChatGPT, Claude, Gemini, DeepSeek). Bug-Fixes und Architektur-Konsolidierung. Erweiterung auf 32 Fenster, Kostenberechnung, Hydraulik, Eigendevis.

Ueberlegung: Geschwindigkeit wichtiger als marginale Qualitaetsverbesserung. 3B bei 90% der Faelle ausreichend, 8B fuer die schwierigen 10%. Parallelisierung bringt mehr als groesseres Modell.

KAPITEL 7

Architektur-Entscheidungen

Fuer jede kritische Entscheidung: Frage, Optionen, Wahl und Begrueendung

1. Thin-AI: C# fuer Logik, LLM nur fuer Textgenerierung?

Optionen: (A) LLM fuer alles (End-to-End) | (B) LLM nur fuer Klassifikation, Rest deterministisch | (C) Kein LLM

✓ **Gewaehlt: (B) Thin-AI — LLM nur fuer VSA-Code-Klassifikation**

Begrueendung: Messungen muessen reproduzierbar und deterministisch sein. Ein LLM liefert bei gleicher Eingabe unterschiedliche Ergebnisse. Pixelbasierte Berechnungen (Flaeche, Ausdehnung) gehoeren in C#-Code, nicht in ein probabilistisches Modell. Das LLM bringt seinen Mehrwert bei der semantischen Zuordnung (welcher VSA-Code passt zu diesem Schadensbild).

2. Lokale Inferenz statt Cloud-API?

Optionen: (A) OpenAI/Claude API | (B) Lokale Inferenz auf eigener GPU | (C) Hybrid

✓ **Gewaehlt: (B) Vollstaendig lokal auf RTX 5090**

Begrueendung: Dies ist eine Teststation — die lokale Ausfuehrung bietet volle Kontrolle ueber Modelle, Latenz und Kosten. Keine laufenden API-Kosten bei ~3000 Videos. Unabhaengigkeit von Internetverbindung. Moeglichkeit, Modelle schnell zu wechseln und zu experimentieren.

3. Ollama statt vLLM oder llama.cpp direkt?

Optionen: (A) vLLM | (B) Ollama | (C) llama.cpp direkt | (D) TensorRT-LLM

✓ **Gewaehlt: (B) Ollama**

Begrueendung: Ollama bietet die einfachste API (REST), automatisches Modell-Management, Keep-Alive-Steuerung und Multi-Modell-Support. vLLM waere leistungsaehiger fuer High-Throughput, aber der Overhead lohnt sich nicht fuer einen Single-User. Ollama hat Polly-kompatible Retry-Logik und Circuit-Breaker im C#-Client.

4. C# + Python Sidecar statt reines Python?

Optionen: (A) Alles in Python | (B) C# Hauptapp + Python Sidecar fuer Vision | (C) Alles in C#

✓ **Gewaehlt: (B) C# WPF-App + Python FastAPI Sidecar**

Begruendung: WPF bietet die beste Desktop-UI-Erfahrung unter Windows. YOLO, DINO und SAM haben das beste Python-Oekosystem. Der Sidecar laeuft als FastAPI-Server auf Port 8100 und wird per HTTP angesprochen. Klare Trennung: C# fuer UI/Logik/Orchestrierung, Python fuer GPU-Inferenz.

5. Temporal Voting statt ByteTrack/OC-SORT?

Optionen: (A) ByteTrack | (B) OC-SORT | (C) Temporal Voting | (D) DeepSORT

✓ **Gewaehlt: (C) Temporal Voting**

Begruendung: Kanalinspektionsvideos haben niedrige Frame-Raten (alle 1.5-3 Sekunden ein Frame). IoU-basierte Tracker (ByteTrack, OC-SORT) brauchen hohe Frame-Raten fuer zuverlaessiges Matching. Bei so grossen Zeitabstaenden bewegt sich die Kamera zu weit. Temporal Voting (Abstimmung ueber N Frames) ist robuster und einfacher.

6. nomic-embed-text beibehalten oder wechseln?

Optionen: (A) nomic-embed-text behalten | (B) NV-CLIP fuer Bild-Embeddings | (C) OpenAI text-embedding-3

✓ **Gewaehlt: (A) nomic-embed-text beibehalten**

Begruendung: Das Modell ist zu tief in KB, QualityGate und WeightLearning integriert. Ein Wechsel wuerde alle Embeddings in der KB invalidieren und ein Re-Embedding aller Samples erfordern. nomic-embed-text liefert gute Ergebnisse bei minimalem VRAM (~0.3 GB). NV-CLIP als Ergaenzung (nicht Ersatz) ist fuer spaeter geplant.

7. Dual-Model Eskalation (3B → 8B) statt einzelнем grossem Modell?

Optionen: (A) Nur 32B (beste Qualitaet) | (B) Nur 3B (schnellstes) | (C) Kaskade 3B → 8B

✓ **Gewaehlt: (C) Kaskade 3B → 8B mit QualityGate**

Begruendung: ~80-90% der Frames sind einfache Faelle (kein Schaden oder klarer Schaden), fuer die 3B ausreicht. Nur Yellow/Red-Faelle eskalieren zu 8B. Das spart ~3 GB VRAM permanent und reduziert die durchschnittliche Latenz von ~1.5s auf ~800ms. Die 8B-Eskalation laeuft nur bei den schwierigen ~10-20% der Faelle.

KAPITEL 8

Audit-Ergebnisse

Findings aus externem Multi-KI-Audit und interner Verifizierung

8.1 Externe Findings (ChatGPT / Gemini Audit)

Die Codebasis wurde durch mehrere KI-Systeme (ChatGPT, Gemini, DeepSeek) extern auditiert. Anschliessend wurden alle Findings von Claude verifiziert. Ergebnis: Mehrere behauptete Bugs waren falsch.

BEHAUPTUNG	STATUS	DETAILS
Fehlende Retry-Logik bei Ollama	FALSCH	OllamaClient hat Polly-Pipeline mit 3 Retries und Circuit Breaker. Vollstaendig implementiert.
Off-by-One bei Frame-Sampling	NICHT GEFUNDEN	Kein Off-by-One-Fehler im Frame-Sampling-Code nachweisbar.
Deadlock bei Pause/Resume	NICHT GEFUNDEN	Pause/Resume-Mechanismus korrekt implementiert mit CancellationToken.
Race Condition TryConsumeBudget	NICHT GEFUNDEN	Budget-Verbrauch ist thread-safe implementiert.

Fazit externe Audits: Von vier behaupteten schwerwiegenden Bugs waren alle vier entweder falsch oder nicht nachweisbar. Dies unterstreicht, dass KI-Audits immer manuell verifiziert werden muessen.

8.2 Verifizierte Findings (Claude-Analyse)

Die folgende Liste enthaelt alle tatsaechlich verifizierten Findings, sortiert nach Schweregrad.

KRITISCH — Sofort beheben (< 1 Tag)

N7**SemaphoreSlim(2,2) in EnhancedVisionAnalysisService.cs:182****KRITISCH**

Erlaubt 2 gleichzeitige 32B-Inferenzen = Out-of-Memory. Loest sich mit 3B→8B-Architektur, aber bis dahin auf SemaphoreSlim(1,1) setzen.

N2 **VsaCodeResolver.NormalizeClock() fehlt Englisch-Mapping** **KRITISCH**
 DINO gibt englische Positionsangaben aus (top/bottom/left/right), aber NormalizeClock erwartet deutsche Begriffe. Uhrlage-Zuordnung fehlerhaft bei DINO-Ergebnissen.

T2 **PdfProtocolTableParser.cs:45 StufeRegex nur [1-4]** **KRITISCH**
 Die Regex fuer Schweregrad-Erkennung im PDF-Import akzeptiert nur Stufen 1-4. Stufe 5 (schwerstes Ausmass) wird nicht erkannt und geht verloren.

HOCH — Kurzfristig beheben (1-3 Tage)

N5 **QuickScanService nutzt FrameQualityFilter nicht** **HOCH**
 Beim Quick Scan werden alle Frames an Qwen geschickt, ohne vorherige Qualitaetsfilterung. 50-70% der Qwen-Inferenzen sind auf qualitativ schlechten Frames verschwendet (unscharf, zu dunkel, Wasser).

N9 **QualityGateService renormalisiert bei fehlenden Signalen** **HOCH**
 Wenn nur ein Signal vorhanden ist, kann dieses allein Green erzeugen. Fix: Minimum 2 Signale fuer Green-Bewertung verlangen.

N3 **Meter-Toleranzen inkonsistent** **HOCH**
 Verschiedene Services verwenden unterschiedliche Toleranzen (0.5m, 1.0m, 1.5m) fuer die Meter-basierte Zuordnung. Sollte einheitlich konfiguriert werden.

N15 **FrameQualityFilter verwirft ohne Logging** **HOCH**
 Verworfen Frames werden nicht geloggt. Es ist nicht nachvollziehbar, wie viele und welche Frames gefiltert wurden.

MITTEL — Spaeter adressieren

N1 **Y < 0.30 verwirft Ferndetektionen still. Gewollt, aber Logging fehlt.** **MITTEL**

N4 **FindingKey-Kollision bei Mehrfach-Schaeden (Code|Clock ohne Meter).** **MITTEL**

N8 **PipeImageWidthRatio hardcoded 0.70. Ungenau bei anderen Kamera-Sichtfeldern.** **MITTEL**

N10 **Consensus braucht ≥ 2 Modelle, YOLO allein bei 0.95 wird trotzdem verworfen.** **MITTEL**

8.3 Gesamtbewertung

**8.5****Architektur**

Clean Architecture, klare Schichten, gute Trennung von Concerns. Thin-AI-Prinzip konsequent umgesetzt. MVVM sauber implementiert.

**7.0****Robustheit**

Retry-Logik vorhanden, Circuit Breaker implementiert, Auto-Save funktioniert. Einige Logging-Luecken und Toleranz-Inkonsistenzen. Solide fuer ein Solo-Projekt.

**6.0****Praezision**

QualityGate-Renormalisierung, fehlende Stufe 5, DINO-Englisch-Bug reduzieren die Genauigkeit. Mit den identifizierten Fixes auf 7.5+ verbesserbar.

KAPITEL 9

Optimierungs-Roadmap

Geplante Verbesserungen in priorisierten Phasen

Sofort umsetzen

SAM 2.1 Box-Batching

Sequentielle Box-Verarbeitung durch gebatchten Forward Pass ersetzen. Ca. 10 Zeilen Code-Aenderung, sofort messbare Verbesserung der SAM-Durchsatzrate.

~10 ZEILEN CODE

Eager Ollama Pre-Load

Qwen beim App-Start laden statt lazy beim ersten Bedarf. Spart 30-60 Sekunden bei der ersten Analyse. Nahezu gratis in der Implementierung.

30-60S GESPART

Phasenplan

1

Qwen 3B Benchmark

50-100 Frames mit gleichem Prompt und Few-Shots testen. Messen: VSA-Code-Treffer, JSON-Stabilitaet, Green/Yellow/Red Verteilung. Go/No-Go: Yellow-Rate darf max. 15% steigen gegenueber 8B. Kandidaten: qwen2.5-vl:3b, Phi-4-Vision. **Achtung:** KV-Cache bei Vision-Modellen groesser als bei Text-LLMs (~2-3 GB/Slot statt 1-1.5 GB) wegen visueller Tokens.

2

Producer-Consumer mit 1 Qwen-Slot

Vision-Pipeline laeuft vor, Qwen konsumiert aus Queue. Kein Risiko, da rein sequentieller Qwen-Betrieb, aber entkoppelt von der Vision-Wartezeit. Erwarteter Durchsatz: ~1.5-2x gegenueber aktuellem Stand.

3

2 parallele Qwen-Slots (OLLAMA_NUM_PARALLEL=2)

GPU-Monitoring mit nvidia-smi waehrend Batch. Messen: p50/p95 Latenz, VRAM-Spitzen, Einfluss auf YOLO/DINO/SAM. **Warnung:** GPU-Contention (Speicherbandbreite) kann Vision-Pipeline verlangsamen. Realistisch: ~2-2.5x Durchsatz (nicht 3x oder 5x).

4

8B permanent dazuladen (0-Sekunden-Eskalation)

Nur wenn Phase 1-3 sauber: 3B×2 + 8B permanent laden = ~24-27 GB VRAM. Eskalation in 0 Sekunden statt 30-60 Sekunden Modell-Swap. Neue Kaskade: 3B → 8B (statt 8B → 32B).

Langfristig geplant**NV-CLIP als paralleler Bild-Retrieval**

Hybrid-Ansatz: nomic fuer Text-zu-Text, NV-CLIP fuer Bild-zu-Bild Similarity. Verbessert Few-Shot-Retrieval durch visuellen Vergleich statt nur Textvergleich.

Knowledge Distillation 32B → 3B

Trainieren eines spezialisierten 3B-Modells mit Wissen aus dem 32B-Modell. Braucht tausende verifizierte Green-Samples in der Knowledge Base als Trainingsdaten.

Durchsatz-Erwartungen (korrigiert)

Urspruengliche Schaetzung (Claude): ~5x Durchsatz — **zu optimistisch**

ChatGPT-Korrektur: GPU-Contention, KV-Cache groesser als angenommen, 3B-Qualitaet unklar.

Realistische Erwartung: 2-3x Durchsatz nach Phase 3, mit 0s Eskalation als Bonus in Phase 4.

Was NICHT gemacht wird

- ~~✗ ByteTrack/OC-SORT — IoU-Tracking bei 1.5-3s Frame-Intervall nutzlos. Temporal Voting ist besser.~~
- ~~✗ NVDEC → GPU-Tensor — Bei 720p/PAL alle 1.5-3s irrelevant. Kein 4K, kein 60fps.~~
- ~~✗ Intel NPU fuer Pre-Screening — Pre-Screening kostet <1ms auf CPU. NPU-Overhead > Gewinn.~~
- ~~✗ Sparsity 2:4 — Braucht sparsifizierte Weights. Aufwand >> Gewinn bei YOLO im ms-Bereich.~~
- ~~✗ FP4 fuer YOLO/DINO — Nur fuer Qwen (LLM). BBox-Regression braucht FP16-Praezision.~~

KAPITEL 10

Projekt-Statistiken

SewerStudio KI 4.1 in Zahlen



Codebase-Metriken

METRIK	WERT	ANMERKUNG
C#-Quelldateien	791	Ohne Tests und CLI-Tools
C#-Codezeilen (gesamt)	~142.700	Inkl. Leerzeilen und Kommentare
Python-Codezeilen (Sidecar)	~3.300	FastAPI, YOLO, DINO, SAM
XAML-Dateien (UI)	32+	WPF Views und Themes
ViewModels	33	MVVM-Pattern mit CommunityToolkit.Mvvm
Services (Application-Schicht)	15+	Interfaces und DTOs
Services (Infrastructure-Schicht)	30+	Import, Export, KI, Kosten

METRIK	WERT	ANMERKUNG
KI-Services (UI-Schicht)	40+	Pipeline, Training, Monitoring
CLI-Tools	13	Eigenstaendige Console-Apps
Unit-Test-Projekte	2	Infrastructure.Tests, Pipeline.Tests

Technologie-Stack

KATEGORIE	TECHNOLOGIE	VERSION / DETAILS
Framework	.NET 8+	LTS, WPF Desktop
UI-Framework	WPF (Windows Presentation Foundation)	MVVM mit CommunityToolkit.Mvvm
Datenbank	SQLite	Knowledge Base, Projektdaten
Video	LibVLC / LibVLCSharp	Plattformunabhaengiger Video-Player
PDF-Generierung	QuestPDF	Programmatische PDF-Erstellung
Excel-Export	ClosedXML / EPPlus	XLSX-Generierung
KI-Inference	Ollama	Lokale LLM-Inference (Qwen, nomic)
Vision-Inference	Python FastAPI Sidecar	YOLO, DINO, SAM auf Port 8100
GPU-Optimierung	TensorRT FP16	YOLO-Beschleunigung
Resilience	Polly	Retry-Policies, Circuit Breaker
HTTP	HttpClient + Polly Pipeline	Kommunikation mit Ollama & Sidecar
Build	MSBuild / dotnet CLI	dotnet build AuswertungPro.sln

Architektur-Projekte

PROJEKT	SCHICHT	BESCHREIBUNG
AuswertungPro.Next.Domain	Domain	Entitaeten, Value Objects, Geschaeftsregeln
AuswertungPro.Next.Application	Application	Use Cases, Interfaces, Service-Vertraege
AuswertungPro.Next.Infrastructure	Infrastructure	Importer, Exporter, DB, externe Integration

PROJEKT	SCHICHT	BESCHREIBUNG
AuswertungPro.Next.UI	Praesentation	WPF Views, ViewModels, Themes, KI-Services
AuswertungPro.Wpf	Legacy	Aeltere WPF-Komponenten (Migration)
*.Tests	Test	Unit Tests fuer Infrastructure und Pipeline

Bemerkenswert: Dieses Projekt — mit seinen ~142.700 Zeilen C#, 32 Fenstern, 140+ KI-Modulen und einer Multi-Modell-Pipeline — wurde von einem einzelnen Entwickler ohne formale Programmierausbildung in ca. 4 Monaten erstellt. Es demonstriert, was mit modernen KI-Assistenten als Entwicklungswerkzeug moeglich ist.

SewerStudio KI 4.1 — System-Audit — Pascal Aschwanden — 12. April 2026

Erstellt mit KI-Unterstuetzung (Claude Opus 4.6). Alle technischen Angaben basieren auf Quellcode-Analyse.